

重 庆 大 学

学 生 实 验 报 告

实验课程名称 数学实验

开课实验室 数学实验教学示范中心

学 院 计算机 年级 大二 专业班 计科 03

学 生 姓 名 严浩睿 学 号 20240395

开 课 时 间 2025 至 2026 学年第 二 学期

总 成 绩	
教师签名	

数 学 与 统 计 学 院 制

开课学院、实验室：LD104

实验时间：2026 年 3 月 14 日

课程名称	数学实验	实验项目名称	MATLAB 软件入门	实验项目类型				
				验证	演示	综合	设计	其他
指导教师	龚勋	成绩				√		

实验目的

- 1. 掌握 MATLAB 数组操作与运算，学习使用 MATLAB 进行简单的图形绘制
- 2. 掌握 MATLAB 二维、三维图形绘制的常用函数，能够实现函数可视化与图形美化
- 3. 了解 MATLAB 脚本文件和函数文件的使用，能够编写自定义函数实现指定功能
- 4. 学会利用 MATLAB 解决数学图形绘制和简单算法验证的实际问题

基础实验 1

问题重述

完成 MATLAB 图形绘制的 1-5、9（1）（2）（4）、10 题，包括同一坐标系绘制正弦余弦曲线、极坐标、隐函数和参数方程曲线绘制；不同坐标系下的图像显示、灰度图直方图绘制、晶体管 I-V 曲线绘制、指定三维曲面绘制、随机曲面插值与等高线绘制等内容，掌握 MATLAB 二维、三维图形绘制的核心函数与图形美化方法。

实验过程

1. 同一坐标系绘制正弦波和余弦波

MATLAB 代码如下：

```
% 新建图形窗口
figure;

% 生成 0 到 2pi 的时间向量，取 1000 个点保证曲线平滑
t = linspace(0,2*pi,1000);

% 绘制正弦曲线
plot(t,sin(t));

% 保持图形，用于叠加绘制
hold on;

% 绘制红色虚线的余弦曲线
plot(t,cos(t),'r--');
```

```

% 图形标注
xlabel('t');
ylabel('y');
title('一个周期内的正弦波和余弦波');
legend('Sin(t)', 'Cos(t)');
% 设置坐标轴范围
xlim([0, 2*pi]);
ylim([-1.4, 1.4]);
% 关闭图形保持
hold off;

```

程序说明：使用 linspace 生成等间距采样点，hold on/off 实现多曲线叠加，通过字符串指定线条样式，xlim/ylim 手动设置坐标轴范围，xlabel/ylabel/title/legend 完成图形标注。

2. 极坐标、隐函数、参数方程曲线

MATLAB 代码如下：

```

% 新建图形窗口，1x3 子图布局
figure; % 子图 1: 极坐标曲线
subplot(1, 3, 1);
theta = linspace(0, 2*pi, 1000);
rho = sin(2*theta).*cos(2*theta);
% 四叶玫瑰线极坐标方程
polarplot(theta, rho);
title('极坐标曲线（四叶玫瑰线）');
% 子图 2: 隐函数曲线
x*sinx + y*siny = 0 subplot(1, 3, 2);
f = @(x, y) x.*sin(x) + y.*sin(y);
fimplicit(f, [-30, 30, -30, 30]);
xlabel('x');
ylabel('y');

```

```

title(' 隐函数曲线  $x \cdot \sin x + y \cdot \sin y = 0$  ');
% 子图 3: 参数方程曲线
subplot(1,3,3);
% 定义参数 t 的范围
t = linspace(0,10*pi,1000);
x = t.*sin(t);
y = t.*cos(t);
fplot(x,y);
xlabel(' x ');
ylabel(' y ');
title(' 参数方程曲线 ');

```

程序说明： subplot(m,n,p)实现 m 行 n 列第 p 个子图绘制； polarplot 绘制极坐标曲线； fimplicit 绘制隐函数曲线，需定义匿名函数并指定变量范围； fplot 绘制参数方程曲线，支持以参数定义的 x、y 函数。

3. 2x2 子图不同坐标系显示图像

MATLAB 代码如下：

```

% 加载图像数据
load mitMap.mat;
% 新建图形窗口，2x2 子图布局
figure;
% 子图 1: square 坐标轴
subplot(2,2,1);
imagesc(mit);
colormap(cMap);
axis square;
title(' 坐标轴: square ');
colorbar;
% 子图 2: tight 坐标轴
subplot(2,2,2);

```

```

imagesc(mit);
colormap(cMap);
axis tight;
title('坐标轴: tight');
colorbar;
% 子图 3: equal 坐标轴
subplot(2,2,3);
imagesc(mit);
colormap(cMap);
axis equal;
title('坐标轴: equal');
colorbar;
% 子图 4: xy 坐标系 (原点在左下角)
subplot(2,2,4);
imagesc(mit);
colormap(cMap);
axis xy;
title('坐标系: xy');
colorbar;

```

程序说明: load 加载.mat 数据文件, imagesc 显示图像矩阵, colormap 设置颜色表, axis square/tight/equal/xy 分别设置不同的坐标轴模式, colorbar 添加颜色刻度条。

4. 灰度图 camera.gif 的灰度分布直方图

MATLAB 代码如下:

```

% 读取灰度图
A = imread('camera.gif');
% 将图像矩阵重塑为一维向量, 便于统计灰度
A_1d = reshape(A,1,[]);
% 新建图形窗口

```

```
figure;
% 绘制灰度直方图
imhist(A);
xlabel('灰度值 (0-255)');
ylabel('像素个数');
title('camera.gif 灰度分布直方图');
```

程序说明：imread 读取图像文件，reshape 将二维图像矩阵转为一维向量，imhist 专门用于绘制图像灰度直方图，直接统计 0-255 各灰度值的像素数量。

5. 晶体管 I-V 特性曲线绘制

MATLAB 代码如下：

```
% 定义参数
VT = 1; % 阈值电压
K = 50e-6; % 常数, 50  $\mu$  A/V2
VDS = linspace(0,5,1000); % 漏源电压 0-5V
VGS = [0,1,2,3,4,5]; % 栅源电压取值
% 构造网格矩阵, 实现矢量化计算
[VDS_mat,VGS_mat] = meshgrid(VDS,VGS);
% 定义逻辑变量, 区分晶体管工作区域
% 截止区: VGS < VT
cutoff = VGS_mat < VT;
% 线性区: VGS >= VT 且 VDS <= VGS - VT
linear = (VGS_mat >= VT) & (VDS_mat <= VGS_mat - VT);
% 饱和区: VGS >= VT 且 VDS > VGS - VT
saturation = (VGS_mat >= VT) & (VDS_mat > VGS_mat - VT);
% 初始化漏源电流矩阵
IDS = zeros(size(VDS_mat));
% 计算各区域电流
IDS(linear)=K.*((VGS_mat(linear)-VT).*VDS_mat(linear)-
0.5*VDS_mat(linear).^2);
```

```

IDS(saturation) = 0.5*K.*(VGS_mat(saturation)-VT).^2;
% 新建图形窗口
figure;
% 绘制不同 VGS 的 I-V 曲线
plot(VDS, IDS');
xlabel('VDS(V)');
ylabel('IDS(μA)');
title('晶体管 I-V 特性曲线');
% 生成图例字符串
legend_str=arrayfun(@(v) ['VGS=', num2str(v), 'V'], VGS, 'UniformOutput', false);
legend(legend_str);
grid on; % 添加网格

```

程序说明：使用 meshgrid 构造栅源和漏源电压的网格矩阵，实现矢量化计算（无循环）；通过逻辑变量区分截止、线性、饱和区，分别按公式计算电流；arrayfun 生成动态图例，grid on 添加网格提升可读性。

6. 三维曲面绘制

MATLAB 代码如下：

```

% 新建图形窗口，3 个子图分别绘制不同曲面
figure;
% 9(1):  $z = x^2 + y^2$  抛物面
subplot(1,3,1);
[x1,y1] = meshgrid(-2:0.1:2,-2:0.1:2);
z1 = x1.^2 + y1.^2;
surf(x1,y1,z1);
shading interp;
% 插值着色，使曲面平滑
colormap(hsv);
xlabel('x');

```

```

ylabel('y');
xlabel('z');
title('抛物面  $z = x^2 + y^2$ ');
colorbar;

% 9(2): 环面
subplot(1,3,2);
[theta,phi] = meshgrid(0:0.1:2*pi,0:0.1:2*pi);
R = 2; % 大半径
r = 1; % 小半径
x2 = (R + r*cos(phi)).*cos(theta);
y2 = (R + r*cos(phi)).*sin(theta);
z2 = r*sin(phi);
surf(x2,y2,z2);
shading interp;
colormap(hsv);
xlabel('x');
ylabel('y');
xlabel('z');
title('环面');
colorbar;

% 9(4):  $z = y^2$  绕 z 轴的旋转面
subplot(1,3,3);
[theta3,r3] = meshgrid(0:0.1:2*pi,0:0.1:2);
x3 = r3.*cos(theta3);
y3 = r3.*sin(theta3);
z3 = y3.^2;
surf(x3,y3,z3);
shading interp;
colormap(hsv);

```

```

xlabel('x');
ylabel('y');
zlabel('z');
title('z = y^2 绕 z 轴旋转面');
colorbar;
% 开启三维旋转
rotate3d on;

```

程序说明：meshgrid 生成三维曲面的平面网格，surf 绘制三维曲面；旋转面采用极坐标转直角坐标实现，shading interp 实现插值着色，rotate3d on 允许鼠标拖动旋转三维图形；环面按极坐标公式生成，区分大小半径。

7. 随机曲面插值与等高线绘制

MATLAB 代码如下：

```

% 新建图形窗口
figure;

% a. 生成 5x5 随机矩阵 Z0，范围[0,1]
Z0 = rand(5,5);

% b. 生成原始网格 X0,Y0
[X0,Y0] = meshgrid(1:5,1:5);

% c. 生成插值后的精细网格 X1,Y1，步长 0.1
[X1,Y1] = meshgrid(1:0.1:5,1:0.1:5);

% d. 三次插值得到 Z1
Z1 = interp2(X0,Y0,Z0,X1,Y1,'cubic');

% e. 绘制插值后的曲面，设置着色和颜色表
surf(X1,Y1,Z1); colormap(hsv);
shading interp;

% f. 保持图形，绘制 15 条等高线
hold on;
contour(X1,Y1,Z1,15);

% g. 添加颜色条

```

```

colorbar;
% h. 设置颜色轴范围 0-1
caxis([0,1]);
% 图形标注
xlabel('x');
ylabel('y');
zlabel('z');
title('随机曲面三次插值与等高线');
hold off;
% 开启三维旋转
rotate3d on;

```

程序说明：rand 生成随机矩阵，interp2 实现二维三次插值，将 5x5 的粗糙网格插值为精细网格；surf 绘制三维曲面，contour 绘制等高线并指定条数，caxis 设置颜色轴的数值范围，保证颜色映射与原始随机值一致。

实验结果及分析

1. **正弦余弦曲线：**成功在同一坐标系绘制出平滑的正弦曲线和红色虚线余弦曲线，坐标轴范围精准，标注清晰，曲线无锯齿，符合实验要求。

2. **1x3 子图曲线：**极坐标子图显示四叶玫瑰线形态规则；隐函数曲线在 $[-30, 30]$ 范围内呈现对称的振荡形态；参数方程曲线为螺旋线，随 t 增大向外延伸，三个子图布局整齐，曲线绘制准确。

3. **2x2 图像坐标系：**四个子图分别按 square/tight/equal/xy 模式显示图像，square/equal 保证坐标轴比例一致，tight 贴合图像边界，xy 模式将图像原点从左上角（ij 模式）移至左下角，图像上下翻转，颜色表和颜色条显示正常。

4. **灰度直方图：**成功绘制 camera.gif 的灰度直方图，横坐标为 0-255 灰度值，纵坐标为对应像素数，可清晰观察到图像的灰度分布特征，比如灰度集中在某一区间，reshape 实现了图像矩阵的维度转换，不影响统计结果。

5. **晶体管 I-V 曲线：**绘制出 6 条不同 VGS 的 I-V 曲线，VGS=0/1V 时为截止区，VGS $\geq$ 2V 时先呈线性增长（线性区），后趋于水平（饱和区），曲线形态与晶体管工作原理完全一致，矢量化计算无循环，运行效率高。

6. **三维曲面**：抛物面呈碗状， z 值随 x 、 y 绝对值增大而增大；环面为轮胎状，三维形态规则； $z=y^2$ 旋转面为旋转抛物面，绕 z 轴对称，三个曲面均采用插值着色，表面平滑，无网格锯齿，rotate3d 可灵活观察曲面不同角度。

7. **随机曲面与等高线绘制**：插值后的随机曲面平滑连续，无明显突变，15 条等高线均匀分布在曲面下方，颜色条范围为 0-1，与原始随机值一致，等高线的疏密反映了曲面的坡度变化，坡度大的区域等高线密集，反之则稀疏。

基础实验 2

问题重述

完成 MATLAB 脚本文件与函数文件的 2、6 (2) (5) 题，包括：编写 getCircle 函数实现圆的坐标计算，编写 concentric.m 和 olympic.m 脚本绘制同心圆和奥运五环；编程验证梅森猜想，即当 n 为素数时， 2^n-1 是否均为素数。编程找出 1000 以内的孪生素数，掌握 MATLAB 函数文件的编写、调用及素数相关的算法实现。

实验过程

1. 圆的绘制函数与脚本

(1) getCircle 函数：

a. 代码如下：

```
function [x,y] = getCircle(center,r)
t = linspace(0,2*pi,1000); % 圆的参数方程，平移至指定圆心
x = center(1) + r*cos(t);
y = center(2) + r*sin(t);
end
```

b. 函数说明：

定义函数输入为圆心（2 维向量）和半径，输出为圆的 x 、 y 坐标；通过圆的参数方程实现，将原点圆平移至指定圆心，linspace 生成足够的采样点保证圆的平滑性。

(2) concentric.m 脚本

a. 代码如下：

```
figure;
center = [0,0];
```

```

r_list = 1:5;
% 获取 jet 色图的 5 种颜色
c_map = jet(5);
% 循环绘制每个圆
for i = 1:length(r_list)
    [x,y] = getCircle(center,r_list(i));
    plot(x,y,'LineWidth',2,'Color',c_map(i,:));
    hold on;
end
axis equal;
xlabel('x');
ylabel('y');
title('五层同心圆 (jet 色图)');
hold off;

```

b. 函数说明:

调用 getCircle 函数获取每个圆的坐标, jet(5)生成 5 种 jet 色系颜色, for 循环实现多层圆绘制, LineWidth 设置线宽为 2, axis equal 保证坐标轴比例一致, 避免圆被拉伸为椭圆。

(3) olympic.m 脚本

a. 代码如下:

```

figure;
center1 = [0,0]; % 蓝环
center2 = [2,0]; % 黑环
center3 = [4,0]; % 红环
center4 = [1,-1]; % 黄环
center5 = [3,-1]; % 绿环
r = 1;
% 绘制五个环, 指定颜色和线宽
[x1,y1] = getCircle(center1,r);

```

```

plot(x1,y1,'b','LineWidth',2);hold on;
[x2,y2] = getCircle(center2,r);
plot(x2,y2,'k','LineWidth',2);
[x3,y3] = getCircle(center3,r);
plot(x3,y3,'r','LineWidth',2);
[x4,y4] = getCircle(center4,r);
plot(x4,y4,'y','LineWidth',2);
[x5,y5] = getCircle(center5,r);
plot(x5,y5,'g','LineWidth',2);
% 图形设置
axis equal;
xlabel('x');
ylabel('y');
title('奥运五环');
hold off;
% 调整坐标轴范围，适配五环
xlim([-1,5]);
ylim([-2,1]);

```

b. 函数说明:

按奥运五环的经典布局定义五个圆心，半径统一为 1；调用 getCircle 函数分别获取每个环的坐标，通过颜色字符指定五环颜色，线宽设为 2，调整坐标轴范围使五环完整显示且布局美观。

2. 数论猜想验证与孪生素数查找

(1) 验证梅森猜想

a. 代码如下:

```

% 验证 n 取 2-20 的素数
n_list = primes(20);
fprintf('验证 20 以内素数的梅森数 (2^n-1) 是否为素数: \n');

```

```

for n = n_list
    mason_num = 2^n - 1;
    is_prime = isprime(mason_num);
    fprintf('n=%d, 梅森数=%d, 是否为素数: %s\n', n, mason_num, is_prime?'是':'否');
end
fprintf('梅森猜想验证结论: 20 以内存在素数 n 使 2^n-1 为合数, 猜想不成立\n');

```

b. 程序说明:

primes(n) 获取 n 以内的所有素数, 2^n-1 计算梅森数, isprime 判断一个数是否为素数; 通过循环验证 20 以内所有素数对应的梅森数, 输出每个梅森数的素性结果

(2) 查找 1000 以内的孪生素数

a. 代码如下:

```

% 获取 1000 以内的所有素数
prime_list = primes(1000);
twin_primes = [];
% 循环判断相邻素数的差是否为 2
for i = 1:length(prime_list)-1
    p1 = prime_list(i);
    p2 = prime_list(i+1);
    if p2 - p1 == 2 twin_primes = [twin_primes; [p1, p2]];
end
end
fprintf('1000 以内的孪生素数对: \n');
disp(twin_primes);
fprintf('1000 以内共有%d 对孪生素数\n', size(twin_primes, 1));

```

b. 程序说明:

primes(1000) 直接获取 1000 以内所有素数, 避免逐个判断的低效率; 循环遍历素数列表, 判断相邻两个素数的差是否为 2, 若为 2 则为孪生素数对, 存入矩阵; size(twin_primes, 1) 统计孪生素数对的数量。

实验结果及分析

1. 圆的绘制函数与脚本

(1) **getCircle 函数**: 函数调用正常, 输入任意圆心和半径均可返回平滑的圆坐标, 如 `getCircle([1,2],3)` 可生成圆心 (1,2)、半径 3 的圆坐标, 无语法错误和逻辑错误, 可重复调用。

(2) **同心圆绘制**: 成功绘制五层以原点为圆心、半径 1-5 的同心圆, 分别采用 jet 色图的 5 种颜色, 线宽 2, 坐标轴等比例, 圆形态规则无畸变, 五层圆层层嵌套, 布局整齐。

(3) **奥运五环绘制**: 成功绘制奥运五环, 五个环按经典布局排列 (上三下二), 颜色分别为蓝、黑、红、黄、绿, 线宽 2, 坐标轴等比例, 五环无畸变, 完整显示在窗口中, 符合奥运五环的视觉特征。

2. 数论猜想验证与孪生素数查找

(1) 梅森猜想验证:

20 以内的素数为 [2, 3, 5, 7, 11, 13, 17, 19], 对应的梅森数及素性结果为:

$n=2$, 梅森数 = 3, 是素数;

$n=3$, 梅森数 = 7, 是素数;

$n=5$, 梅森数 = 31, 是素数;

$n=7$, 梅森数 = 127, 是素数;

$n=11$, 梅森数 = 2047, 否 ($2047=23 \times 89$);

$n=13$, 梅森数 = 8191, 是素数;

$n=17$, 梅森数 = 131071, 是素数;

$n=19$, 梅森数 = 524287, 是素数。**结论**: 当 $n=11$ (素数) 时, 梅森数 2047 为合数, 因此梅森猜想不成立。

(2) 1000 以内的孪生素数:

成功查找出 1000 以内的所有孪生素数对, 前几对为 (3, 5)、(5, 7)、(11, 13)、(17, 19)……, 最后几对为 (983, 985) (非素数)、(991, 993) (非素数)、(997, 999) (非素数), 总计有 **35 对孪生素数**。结果符合孪生素数的定义, 无遗漏和错误, 程序通过 `primes` 函数直接获取素数列表, 运行效率远高于逐个判断 1-1000 的数是否为素数。

3. 调试情况记录

(1) 编写 `getCircle` 函数时, 初期未将函数保存为 `getCircle.m` (与函数名一致), 导致调用时报错, 按 MATLAB 函数命名规则修正后调用正常。

(2) 绘制奥运五环时, 初期圆心坐标设置不合理导致五环重叠, 调整圆心水平间距为 2、垂直间距为 1 后, 布局符合要求。

(3) 验证梅森猜想时，初期直接遍历 1-20 的数而非素数，导致计算冗余，使用 `primes(20)` 获取素数列表后，简化了循环过程。

(4) 查找孪生素数时，初期未考虑 `primes` 函数的便捷性，逐个判断数的素性导致运行较慢，改用 `primes(1000)` 后运行效率大幅提升。

教师签名

年 月 日